



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Veriicación de Tipos

Práctica 6

Presenta:
Ramírez Juárez María Fernanda - 321204747
Rojo Peña Manuel Ianluck - 118005762

Profesor:
Pablo Martínez Yessica Janeth

Ayudante de laboratorio:
Salazar González Pedro Yamil

Grupo: 7010, 2027-1

Fecha de entrega:
1 de noviembre, 2026

Objetivo

Hasta la Práctica 5 el compilador de MINIC sabía responder *¿qué elementos forman el programa?*, *¿está bien escrito?* y *¿cada nombre que se usa fue declarado y es visible?*, pero todavía hay programas que pasan todas esas revisiones y que no tienen sentido. Pensemos en `int x; bool activo; x = activo;`: cada token está en una posición válida, ambas variables existen y no hay redeclaraciones, y aun así la asignación intenta guardar un booleano en una variable entera. Solo el análisis de tipos puede concluir que esa asignación es incorrecta.

En esta práctica extendimos el analizador semántico para que determine el tipo de cada expresión y verifique la compatibilidad de tipos en inicializaciones, asignaciones, operadores y condiciones. Como en las prácticas anteriores, la entrada del verificador sigue siendo el AST, no vuelve a leer el archivo ni a consumir tokens, y los tipos declarados los obtiene de la tabla de símbolos de la Práctica 5.

Concretamente, nuestros objetivos fueron:

1. Representar los tipos de MINIC (`int` y `bool`) con una representación común y una función que produzca nombres legibles para los diagnósticos.
2. Determinar el tipo de literales, identificadores y expresiones compuestas, propagándolo de las hojas hacia la raíz del árbol.
3. Verificar inicializaciones, asignaciones, los cuatro grupos de operadores binarios, la negación unaria y las condiciones de `if` y `while`.
4. Reportar todas las incompatibilidades con línea, columna, operador o construcción involucrada, tipo esperado y tipo encontrado.
5. Evitar los errores en cascada con `TYPE_ERROR`, sin ocultar errores que sean independientes.
6. Conservar todo lo anterior (análisis léxico, sintáctico, ámbitos, *shadowing*) y mantener un único propietario para cada recurso, sin fugas de memoria.

Evolución respecto de la Práctica 5

Módulos modificados o agregados

Antes de entrar en los detalles, veamos qué tocamos y qué dejamos igual:

- `type.h` y `type.c`: **módulos nuevos**. Contienen la enumeración `Type` (que antes vivía en `symbol.h`), `type_to_string` y `types_are_compatible`.
- `type_checker.h` y `type_checker.c`: **módulos nuevos**. Son el verificador de tipos: determinan el tipo de las expresiones, aplican las reglas de cada operador y comprueban inicializaciones, asignaciones y condiciones.

- `semantic.h` y `semantic.c`: **modificados**. Siguen recorriendo las sentencias, declarando símbolos y resolviendo nombres, pero ahora delegan la verificación de tipos al nuevo módulo en el mismo recorrido. Además alojan los diagnósticos compartidos.
- `ast.h` y `ast.c`: **modificados**. Cada nodo tiene ahora un campo `inferred_type`, que nace en `TYPE_UNKNOWN`.
- `symbol.h`: **modificado**. Ya no define `Type`; lo toma de `type.h`.
- `Makefile`: **modificado**. Incorpora los módulos nuevos y las pruebas públicas de esta práctica, y retira las de las prácticas anteriores.
- `main.c`, el lexer, el parser, `scope.*`, `symbol.c` y `symbol_table.*` se conservan sin cambios.

Es decir, **extendimos la Práctica 5 sin cambiar MINIC** : ninguna palabra reservada, regla gramatical ni estructura del AST (más allá del campo del tipo inferido) se modificó.

Arquitectura actualizada

```
main
 |
 v
Lexer ----> Parser ----> AST
                          |
                          v
                  Semantic Analyzer  (semantic.c)
                   |-- recorrido de sentencias
                   |-- declaraciones, ambitos y resolucion de nombres
                   |-- diagnosticos compartidos
                   |
                   |   delega, en el mismo recorrido
                   v
                  Type Checker       (type_checker.c)   <- NUEVO
                   |-- tipo de cada expresion
                   |-- reglas por operador
                   |-- inicializaciones, asignaciones, condiciones
                   '-- propagacion de TYPE_ERROR
                          |
                          v
                    Symbol Table  ---->  Type (type.c)
```

Seguimos con la regla de siempre: cada módulo tiene una sola responsabilidad. El analizador semántico decide *qué visitar y en qué orden*, y el verificador decide *qué tipo tiene lo visitado*. La tabla de símbolos sigue sin conocer las reglas de tipos, solo guarda el tipo declarado de cada variable.

Cambios en el AST

Para guardar el tipo calculado agregamos un único campo a `ASTNode`, y para poder usar el tipo `Type` dentro del árbol lo importamos desde `type.h`:

```
#include "semantic/type.h"  //Type (type.h no incluye nada del proyecto: sin ciclo)

struct ASTNode {
    ASTNodeType type;
    int line;
    int column;

    // Tipo inferido por el verificador de tipos, todo nodo nace con TYPE_UNKNOWN
    // Solo las expresiones reciben TYPE_INT, TYPE_BOOL o TYPE_ERROR
    Type inferred_type;

    union { /* ... sin cambios ... */ } data;
};
```

Código 1: Cambio en `ast.h`: el nodo guarda su tipo inferido.

Un detalle que nos obligó a pensar los nodos se reservan con `calloc`, que deja todo en cero, y en nuestra enumeración el valor cero es `TYPE_INT`. Sin ninguna corrección, *todo* nodo habría nacido como si fuera entero. Por eso el constructor interno de nodos asigna explícitamente el valor inicial:

```
    node->type = type;
    node->line = line;
    node->column = column;
    node->inferred_type = TYPE_UNKNOWN;  //calloc deja 0 == TYPE_INT
```

Código 2: Cambio en `alloc_node` de `ast.c`.

Como `inferred_type` es un valor (una enumeración) y no un apuntador, `ast_destroy` no necesita ningún cambio.

Cambios en `symbol.h` y en `semantic.h`

`ast.h` necesita `Type`, y `symbol.h` ya incluía a `ast.h`. Si `Type` hubiera seguido en `symbol.h`, tendríamos un ciclo de inclusión. La solución fue sacar `Type` a un encabezado propio, `type.h`, que no incluye ningún otro archivo del proyecto:

```
#include "ast/ast.h"
#include "semantic/type.h"  //Type ya no se define aqui (P6): vive en type.h

typedef struct Symbol {
    char *name;
    Type type;
    /* ... sin cambios ... */
} Symbol;
```

Código 3: Cambio en `symbol.h`: `Type` ya no se define aquí.

En `semantic.h` agregamos tres funciones de diagnóstico públicas, porque ahora las usan tanto `semantic.c` como `type_checker.c`, así el conteo de errores y el formato de los mensajes viven en un solo lugar:

```
void semantic_report_error(SemanticAnalyzer *analyzer, int line, int column,
                           const char *format, ...);
void semantic_report_undeclared(SemanticAnalyzer *analyzer, const char *name,
                                int line, int column);
void semantic_report_internal_error(SemanticAnalyzer *analyzer);
```

Código 4: Declaraciones nuevas en `semantic.h`.

Sistema de tipos de MiniC

Los tipos del lenguaje y su representación interna

MiniC usa tipado estático, el tipo de las variables y de las expresiones se determina durante la compilación, antes de ejecutar nada. Existen dos tipos de datos, `int` y `bool`, y los representamos con una enumeración común:

```
typedef enum {
    TYPE_INT,
    TYPE_BOOL,
    TYPE_ERROR,
    TYPE_UNKNOWN
} Type;

const char *type_to_string(Type type);
int types_are_compatible(Type expected, Type actual);
```

Código 5: Representación de los tipos en `type.h`.

- `TYPE_INT` y `TYPE_BOOL`: expresión o variable entera o booleana.
- `TYPE_ERROR`: expresión cuyo análisis produjo un error semántico. Es lo que permite no repetir diagnósticos (lo veremos en detalle más adelante).
- `TYPE_UNKNOWN`: tipo todavía no determinado o no aplicable; es el valor inicial de todo nodo del AST y también lo que devuelve el verificador para algo que no es una expresión.

Reglas de compatibilidad

La compatibilidad es **exacta**, un valor `int` solo puede asignarse a una variable `int` y un `bool` solo a una `bool`. No hay conversiones implícitas, y por lo mismo un entero distinto de cero *no* es verdadero en una condición. La función `types_are_compatible` lo expresa en una línea; además, los dos tipos especiales nunca son compatibles con nada, para que un `TYPE_ERROR` no se "acepte" por accidente:

```
#include "semantic/type.h"

const char*
```

```
type_to_string(Type type)
{
    switch (type) {
    case TYPE_INT:
        return "int";
    case TYPE_BOOL:
        return "bool";
    case TYPE_ERROR:
        return "error";
    case TYPE_UNKNOWN:
        return "unknown";
    }
    return "?";     //valor fuera del enum
}

int
types_are_compatible(Type expected, Type actual)
{
    return expected == actual
           && expected != TYPE_ERROR
           && expected != TYPE_UNKNOWN;
}
```

Código 6: `type.c` completo.

Dos decisiones pequeñas `type_to_string` no tiene `default`, de modo que si algún día agregamos un tipo, el compilador nos avisa con `-Wswitch` que falta un caso; y si llegara un valor fuera de la enumeración devuelve `"?"` en lugar de un apuntador inválido.

Reglas de tipos por construcción

Estas son las reglas que implementa el verificador, son exactamente las del enunciado:

Construcción	**Operadores / forma**	**Operandos**	**Resultado**
Literal entero	`42`	—	`int`
Literal booleano	`true`, `false`	—	`bool`
Identificador	`x`	—	tipo de su símbolo visible
Aritméticos	`+ - * /`	dos `int`	`int`
Negación unaria	`-`	un `int`	`int`
Relacionales	`< <= > >=`	dos `int`	`bool`
Igualdad	`== !=`	dos del mismo tipo	`bool`
Lógicos	`&& ||`	dos `bool`	`bool`
Inicialización	`T x = e;`	`e` de tipo `T`	—
Asignación	`x = e;`	`e` del tipo de `x`	—
Condición	`if (c)`, `while (c)`	`c` de tipo `bool`	—
Impresión	`print(e);`	`e` válida, de cualquier tipo	—

Table 1: Reglas de tipos de MiniC .

Tampoco evaluamos nada el verificador solo mira tipos, por eso `10 / 0` no produce ningún diagnóstico en esta práctica, y `print` acepta cualquier expresión que sea semánticamente válida.

El verificador de tipos

Estrategia para determinar el tipo de una expresión

Elegimos un recorrido *ascendente* para conocer el tipo de un nodo primero obtenemos el tipo de sus hijos y después aplicamos la regla del operador. Con `5 < 2 + 3`, por ejemplo, los literales son `int`, la suma recibe `int` e `int` y produce `int`, y la comparación recibe `int` e `int` y produce `bool`. Los tipos viajan de las hojas hacia la raíz.

Además tomamos dos decisiones de diseño que vale la pena justificar:

- **Un solo recorrido.** Podríamos haber resuelto los nombres en una pasada y verificado los tipos en otra. Pero el enunciado nos advierte que, con dos pasadas,no podemos consultar solo la tabla final aceptaríamos usos anteriores a la declaración o elegiríamos el símbolo equivocado por culpa del *shadowing*. En un solo recorrido, cada uso se resuelve contra el ámbito que está vigente *en ese momento*, y además un uso no declarado se reporta una sola vez.

- **Los tipos se devuelven *y* se almacenan.** `type_check_expression` devuelve el tipo de la expresión, que es lo que el padre necesita de inmediato, y además lo guarda en `node->inferred_type`, que será útil en etapas posteriores. Solo las expresiones reciben tipo; las sentencias conservan `TYPE_UNKNOWN`.

Interfaz del verificador

```
typedef enum {
    CONDITION_IF,
    CONDITION_WHILE
} ConditionKind;

Type type_from_declared(ASTDeclaredType declared);
Type type_check_expression(SemanticAnalyzer *analyzer, ASTNode *node);
void type_check_initialization(SemanticAnalyzer *analyzer, ASTNode *declaration
    );
void type_check_assignment(SemanticAnalyzer *analyzer, ASTNode *assignment,
                           const Symbol *target);
void type_check_condition(SemanticAnalyzer *analyzer, ASTNode *condition,
                          ConditionKind kind);
```

Código 7: Interfaz pública de `type_checker.h`.

Fijémonos en la división de trabajo: `semantic.c` *resuelve* el destino de una asignación y se lo pasa ya resuelto (`target`) al verificador, que solo compara tipos. Así la resolución de nombres de sentencias sigue siendo responsabilidad del analizador, y el verificador no tiene que volver a reportar una variable no declarada.

El archivo `type_checker.c` está organizado por propósito, lo recorremos completo en las siguientes secciones; todas las funciones auxiliares son `static`.

Prototipos internos y utilidad de propagación

Empezamos con los prototipos del módulo, que ya muestran su estructura: una función de visita por tipo de expresión (`visit_*`), una regla por categoría de operador (`check_*`) y una función por tipo de diagnóstico (`report_*`). Así evitamos concentrar todas las responsabilidades en una función extensa. También aparece `is_undetermined`, la utilidad clave de la propagación, y `type_from_declared`, que traduce el tipo escrito en una declaración al tipo interno.

```c
#include "semantic/type_checker.h"

static int is_undetermined(Type type);

static Type visit_identifier(SemanticAnalyzer *a, ASTNode *node);
static Type visit_integer_literal(SemanticAnalyzer *a, ASTNode *node);
static Type visit_boolean_literal(SemanticAnalyzer *a, ASTNode *node);
static Type visit_unary_expression(SemanticAnalyzer *a, ASTNode *node);
static Type visit_binary_expression(SemanticAnalyzer *a, ASTNode *node);

static Type check_arithmetic(SemanticAnalyzer *a, const ASTNode *node, Type
    left, Type right);
static Type check_relational(SemanticAnalyzer *a, const ASTNode *node, Type
    left, Type right);
static Type check_equality(SemanticAnalyzer *a, const ASTNode *node, Type left,
     Type right);
static Type check_logical(SemanticAnalyzer *a, const ASTNode *node, Type left,
    Type right);

static const char *binary_operator_symbol(BinaryOperator op);
static void report_operand_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                                    Type expected, Type left, Type right);
static void report_equality_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                                     Type left, Type right);
static void report_unary_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                                  Type expected, Type found);
static void report_initialization_mismatch(SemanticAnalyzer *a, int line, int
    column,
                                           Type declared, Type found);
static void report_assignment_mismatch(SemanticAnalyzer *a, int line, int
    column,
                                       Type target, Type found);
static void report_condition_mismatch(SemanticAnalyzer *a, int line, int column
    ,
                                      ConditionKind kind, Type found);

Type
type_from_declared(ASTDeclaredType declared)
{
    return declared == AST_TYPE_BOOL ? TYPE_BOOL : TYPE_INT;
}

/*
 * Un tipo "envenenado" (TYPE_ERROR) o aun sin determinar (TYPE_UNKNOWN, p. ej.
 * tras un fallo interno) nunca produce un diagnostico nuevo: solo se propaga.
 */
static int
is_undetermined(Type type)
```

```
{
    return type == TYPE_ERROR || type == TYPE_UNKNOWN;
}
```

Código 8: `type_checker.c`: prototipos, `type_from_declared` e `is_undetermined`.

`type_from_declared` existía antes, con otro nombre y como función privada de `semantic.c` (`type_from_ast`), la movimos aquí porque ahora la necesitan los dos módulos y porque `type.h` no puede incluir a `ast.h` (ya vimos el ciclo).

El despachador de expresiones

`type_check_expression` es el punto de entrada, despacha según el tipo de nodo a la función de visita correspondiente y, *en un único lugar*, escribe el resultado en `inferred_type`. Si recibe algo que no es una expresión (o si ya hubo un fallo interno) devuelve `TYPE_UNKNOWN` sin tocar el nodo.

```
Type
type_check_expression(SemanticAnalyzer *a,
                      ASTNode *node)
{
    Type result;

    if (a == NULL || node == NULL || a->had_internal_error)
        return TYPE_UNKNOWN;

    switch (node->type) {
    case AST_IDENTIFIER:
        result = visit_identifier(a, node);
        break;
    case AST_INTEGER_LITERAL:
        result = visit_integer_literal(a, node);
        break;
    case AST_BOOLEAN_LITERAL:
        result = visit_boolean_literal(a, node);
        break;
    case AST_UNARY_EXPRESSION:
        result = visit_unary_expression(a, node);
        break;
    case AST_BINARY_EXPRESSION:
        result = visit_binary_expression(a, node);
        break;
    default:            //no es una expresion
        return TYPE_UNKNOWN;
    }

    node->inferred_type = result;
    return result;
}
```

Código 9: `type_checker.c`: `type_check_expression`.

Verificación de inicializaciones, asignaciones y condiciones

Estas tres funciones comparten un patrón que conviene tener presente siempre analizan la expresión primero (para encontrar errores independientes dentro de ella), y solo después deciden si hay incompatibilidad. Si la expresión ya es `TYPE_ERROR` no emiten nada, porque el error de origen ya se reportó.

```
void
type_check_initialization(SemanticAnalyzer *a,
                          ASTNode *declaration)
{
    ASTNode *initializer;
    Type declared;
    Type actual;

    if (a == NULL || declaration == NULL
        || declaration->type != AST_VARIABLE_DECLARATION)
        return;

    initializer = declaration->data.variable_declaration.initializer;
    if (initializer == NULL)
        return;     //declaracion sin inicializador

    //Siempre se analiza: puede tener errores independientes del tipo declarado
    actual = type_check_expression(a, initializer);
    if (is_undetermined(actual))
        return;     //el error de origen ya se reporto

    //Tipo ESCRITO en esta declaracion, no el del simbolo (en una redeclaracion
    puede diferir)
    declared = type_from_declared(declaration->data.variable_declaration.
   declared_type);
    if (!types_are_compatible(declared, actual))
        report_initialization_mismatch(a, initializer->line, initializer->
   column,
                                       declared, actual);
}

void
type_check_assignment(SemanticAnalyzer *a,
                      ASTNode *assignment,
                      const Symbol *target)
{
    Type actual;

    if (a == NULL || assignment == NULL || assignment->type != AST_ASSIGNMENT)
        return;

    //Siempre se analiza el valor, incluso si el destino no existe
    actual = type_check_expression(a, assignment->data.assignment.value);

    if (target == NULL || is_undetermined(actual))
        return;     //uso no declarado o error previo: ya reportado

    if (!types_are_compatible(target->type, actual))
        report_assignment_mismatch(a, assignment->line, assignment->column,
```

```
                                    target->type, actual);
}

void
type_check_condition(SemanticAnalyzer *a,
                     ASTNode *condition,
                     ConditionKind kind)
{
    Type actual;

    if (a == NULL)
        return;

    actual = type_check_expression(a, condition);
    if (is_undetermined(actual))
        return;

    if (actual != TYPE_BOOL)
        report_condition_mismatch(a, condition->line, condition->column, kind,
    actual);
}
```

Código 10: `type_checker.c`: inicializaciones, asignaciones y condiciones.

Comentemos las decisiones, porque cada una responde a una frase concreta del enunciado:

- **Inicialización.** Comparamos el inicializador con el tipo *escrito en la declaración que se está analizando*, no con el del símbolo registrado. Parece lo mismo, pero no lo es cuando hay una redeclaración: en `int x; bool x = true;` la segunda declaración es un error de redeclaración, y su inicializador sigue siendo un `bool` perfectamente válido para el tipo que ahí se escribió. Si comparáramos contra el símbolo original (`int`) reportaríamos un segundo error que no existe.
- **Asignación.** El destino llega ya resuelto. Si es `NULL`, la variable no está declarada: ese error ya se reportó, así que no comparamos contra un tipo inexistente, pero el valor sí se analiza y nunca modificamos `target->type` una asignación incorrecta no cambia el tipo de la variable.
- **Condición.** Solo `TYPE_BOOL` es aceptable, un `int`, aunque sea distinto de cero, no es una condición válida. Como la función no recorre ni el cuerpo ni las ramas, un error aquí no impide analizarlas (eso lo garantiza `semantic.c`).

Visita de cada tipo de expresión

Aquí está el corazón de la regla "el tipo de un identificador es el de su símbolo visible" y de la propagación hacia arriba:

```
static Type
visit_identifier(SemanticAnalyzer *a,
                 ASTNode *node)
{
    const char *name = node->data.identifier.name;
    Symbol *symbol = symbol_table_lookup(&a->symbol_table, name);

```

```
    if (symbol == NULL) {
        semantic_report_undeclared(a, name, node->line, node->column);
        return TYPE_ERROR;
    }
    return symbol->type;
}

static Type
visit_integer_literal(SemanticAnalyzer *a,
                      ASTNode *node)
{
    (void)a;
    (void)node;
    return TYPE_INT;
}

static Type
visit_boolean_literal(SemanticAnalyzer *a,
                      ASTNode *node)
{
    (void)a;
    (void)node;
    return TYPE_BOOL;
}

static Type
visit_unary_expression(SemanticAnalyzer *a,
                       ASTNode *node)
{
    Type operand = type_check_expression(a, node->data.unary.operand);

    if (is_undetermined(operand))
        return TYPE_ERROR;

    //El unico operador unario de MiniC es la negacion aritmetica
    if (operand != TYPE_INT) {
        report_unary_mismatch(a, node, TYPE_INT, operand);
        return TYPE_ERROR;
    }
    return TYPE_INT;
}

static Type
visit_binary_expression(SemanticAnalyzer *a,
                        ASTNode *node)
{
    //Ambos operandos se analizan siempre: sus errores son independientes
    Type left = type_check_expression(a, node->data.binary.left);
    Type right = type_check_expression(a, node->data.binary.right);

    if (is_undetermined(left) || is_undetermined(right))
        return TYPE_ERROR;

    switch (node->data.binary.operator) {
    case OP_ADD:
    case OP_SUBTRACT:
```

```
64      case OP_MULTIPLY:
65      case OP_DIVIDE:
66          return check_arithmetic(a, node, left, right);
67      case OP_LESS:
68      case OP_LESS_EQUAL:
69      case OP_GREATER:
70      case OP_GREATER_EQUAL:
71          return check_relational(a, node, left, right);
72      case OP_EQUAL:
73      case OP_NOT_EQUAL:
74          return check_equality(a, node, left, right);
75      case OP_AND:
76      case OP_OR:
77          return check_logical(a, node, left, right);
78      }
79      return TYPE_ERROR;      //operador fuera del enum
80  }
```

Código 11: `type_checker.c`: visitas por tipo de expresión.

- `visit_identifier` usa `symbol_table_lookup`, la *misma búsqueda visible* de la Práctica 5, por lo que el *shadowing* se respeta sin código adicional. Si el símbolo no existe, reporta la variable no declarada y devuelve `TYPE_ERROR`.
- En `visit_binary_expression`, *ambos* operandos se analizan antes de decidir nada. Si analizáramos el izquierdo y, al ver un error, regresáramos sin visitar el derecho, perderíamos los errores independientes del lado derecho (por ejemplo, otra variable no declarada).
- El `switch` sobre el operador no tiene `default`, para que `-Wswitch` nos avise si agregamos un operador y olvidamos clasificarlo.

Reglas por categoría de operador

Cada categoría tiene su propia función, todas reciben tipos *ya determinados*, porque `visit_binary_expression` filtró antes cualquier `TYPE_ERROR`:

```
1   static Type
2   check_arithmetic(SemanticAnalyzer *a,
3                    const ASTNode *node,
4                    Type left, Type right)
5   {
6       if (left == TYPE_INT && right == TYPE_INT)
7           return TYPE_INT;
8
9       report_operand_mismatch(a, node, TYPE_INT, left, right);
10      return TYPE_ERROR;
11  }
12
13  static Type
14  check_relational(SemanticAnalyzer *a,
15                   const ASTNode *node,
16                   Type left, Type right)
17  {
```

```c
    if (left == TYPE_INT && right == TYPE_INT)
        return TYPE_BOOL;

    report_operand_mismatch(a, node, TYPE_INT, left, right);
    return TYPE_ERROR;
}

static Type
check_equality(SemanticAnalyzer *a,
               const ASTNode *node,
               Type left, Type right)
{
    if (left == right)      //int == int, bool == bool
        return TYPE_BOOL;

    report_equality_mismatch(a, node, left, right);
    return TYPE_ERROR;
}

static Type
check_logical(SemanticAnalyzer *a,
              const ASTNode *node,
              Type left, Type right)
{
    if (left == TYPE_BOOL && right == TYPE_BOOL)
        return TYPE_BOOL;

    report_operand_mismatch(a, node, TYPE_BOOL, left, right);
    return TYPE_ERROR;
}
```

Código 12: `type_checker.c`: reglas por categoría.

Fijémonos en cómo se distinguen las categorías que parecen parecidas, los relacionales y los aritméticos exigen los mismos operandos (`int`), pero producen tipos distintos (`bool` frente a `int`). La igualdad es la única que no exige un tipo concreto sino *el mismo* tipo en ambos lados (y por eso su mensaje es diferente) y los lógicos son los únicos que exigen `bool`.

Diagnósticos de tipos

Todos los mensajes de tipos pasan por `semantic_report_error`, que incrementa el contador y escribe el encabezado con la posición; cada función `report_*` solo arma el texto de su caso:

```c
static const char*
binary_operator_symbol(BinaryOperator op)
{
    switch (op) {
    case OP_ADD:           return "+";
    case OP_SUBTRACT:      return "-";
    case OP_MULTIPLY:      return "*";
    case OP_DIVIDE:        return "/";
    case OP_LESS:          return "<";
    case OP_LESS_EQUAL:    return "<=";
    case OP_GREATER:       return ">";
    case OP_GREATER_EQUAL: return ">=";
```

```
    case OP_EQUAL:          return "==";
    case OP_NOT_EQUAL:      return "!=";
    case OP_AND:            return "&&";
    case OP_OR:             return "||";
    }
    return "?";
}

static void
report_operand_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                        Type expected, Type left, Type right)
{
    semantic_report_error(a, node->line, node->column,
                          "el operador '%s' requiere operandos\n"
                          "de tipo '%s', pero se encontraron\n"
                          "'%s' y '%s'.",
                          binary_operator_symbol(node->data.binary.operator),
                          type_to_string(expected),
                          type_to_string(left), type_to_string(right));
}

static void
report_equality_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                         Type left, Type right)
{
    semantic_report_error(a, node->line, node->column,
                          "el operador '%s' requiere operandos\n"
                          "del mismo tipo, pero se encontraron\n"
                          "'%s' y '%s'.",
                          binary_operator_symbol(node->data.binary.operator),
                          type_to_string(left), type_to_string(right));
}

static void
report_unary_mismatch(SemanticAnalyzer *a, const ASTNode *node,
                      Type expected, Type found)
{
    semantic_report_error(a, node->line, node->column,
                          "el operador unario '-' requiere\n"
                          "un operando de tipo '%s',\n"
                          "pero se encontro '%s'.",
                          type_to_string(expected), type_to_string(found));
}

static void
report_initialization_mismatch(SemanticAnalyzer *a, int line, int column,
                               Type declared, Type found)
{
    semantic_report_error(a, line, column,
                          "no es posible inicializar una variable\n"
                          "de tipo '%s' con una expresion de tipo '%s'.",
                          type_to_string(declared), type_to_string(found));
}

static void
report_assignment_mismatch(SemanticAnalyzer *a, int line, int column,
```

```
                            Type target, Type found)
{
    semantic_report_error(a, line, column,
                          "no es posible asignar un valor de tipo '%s'\n"
                          "a una variable de tipo '%s'.",
                          type_to_string(found), type_to_string(target));
}

static void
report_condition_mismatch(SemanticAnalyzer *a, int line, int column,
                          ConditionKind kind, Type found)
{
    semantic_report_error(a, line, column,
                          "la condicion de una sentencia '%s'\n"
                          "debe ser de tipo 'bool',\n"
                          "pero se encontro '%s'.",
                          kind == CONDITION_IF ? "if" : "while",
                          type_to_string(found));
}
```

Código 13: `type_checker.c`: diagnósticos.

Integración con el analizador semántico

Funciones nuevas o que cambiaron de ámbito

`report_internal_error` y `report_undeclared` eran funciones privadas del archivo, como ahora también las necesita el verificador, las hicimos públicas y les agregamos el prefijo `semantic_` (el cuerpo no cambió, solo el nombre; el mismo renombrado se aplicó en todas las llamadas dentro de `semantic.c`). Lo realmente nuevo es `semantic_report_error`, una función con número variable de argumentos, similar a `printf`, que sirve para todos los diagnósticos de tipos:

```
void
semantic_report_internal_error(SemanticAnalyzer *a)      /* antes:
   report_internal_error */
{
    if (!a->had_internal_error)
        fprintf(stderr,
                "Error interno: no se pudo reservar memoria durante el analisis
    semantico.\n");
    a->had_internal_error = 1;
}

void
semantic_report_error(SemanticAnalyzer *a,               /* NUEVA */
                      int line, int column,
                      const char *format, ...)
{
    va_list args;

    a->error_count++;
    fprintf(stderr, "Error semantico [%d:%d]:\n", line, column);
    va_start(args, format);
    vfprintf(stderr, format, args);
```

```
    va_end(args);
    fputc('\n', stderr);
}

void
semantic_report_undeclared(SemanticAnalyzer *a,          /* antes:
   report_undeclared */
                           const char *name,
                           int line, int col)
{
    a->error_count++;
    fprintf(stderr, "Error semantico [%d:%d]:\n"
                    "la variable '%s' no ha sido declarada.\n",
            line, col, name);
}
```

Código 14: `semantic.c`: funciones de diagnóstico públicas.

Los `...` de `semantic_report_error` importan la declaración en `semantic.h` y la definición tienen que coincidir exactamente. En un primer intento los omitimos en el encabezado y el compilador rechazó todas las llamadas con "too many arguments" (lo comentamos en los problemas encontrados).

Funciones modificadas

Las sentencias ya no recorren expresiones por su cuenta llaman al verificador, la asignación es el cambio más interesante, porque ahora *resuelve* el destino aquí y se lo pasa ya resuelto al verificador:

```
static void
visit_assignment(SemanticAnalyzer *a,
                 ASTNode *node)
{
    Symbol *target = symbol_table_lookup(&a->symbol_table, node->data.
   assignment.name);

    if (target == NULL)
        semantic_report_undeclared(a, node->data.assignment.name, node->line,
   node->column);

    //target puede ser NULL: el verificador analiza el valor pero no compara
   tipos
    type_check_assignment(a, node, target);
}

static void
visit_print_statement(SemanticAnalyzer *a,
                      ASTNode *node)
{
    //print acepta int y bool: solo exige que la expresion sea valida
    (void)type_check_expression(a, node->data.print_statement.expression);
}
```

Código 15: `semantic.c`: `visit_assignment` y `visit_print_statement`.

El `if` (y, igual, el `while`, que solo cambia `CONDITION_IF` por `CONDITION_WHILE` y tiene un único cuerpo) verifica la condición y, sin importar el resultado, sigue con las ramas:

```
static void
visit_if_statement(SemanticAnalyzer *a,
                   ASTNode *node)
{
    type_check_condition(a, node->data.if_statement.condition, CONDITION_IF);
    //un error en la condicion no impide analizar las ramas
    visit_statement(a, node->data.if_statement.then_branch);
    visit_statement(a, node->data.if_statement.else_branch);
}
```

Código 16: `semantic.c`: `visit_if_statement`.

En `visit_variable_declaration` el cambio es doble el tipo del símbolo viene de `type_from_declared`, y al final, en lugar de recorrer el inicializador, se llama al verificador. El resto de la función (búsqueda local, no sobrescribir en una redeclaración, propiedad del símbolo insertado) quedó igual.

```
        Symbol *sym = symbol_create(name,
                                    type_from_declared(node->data.
    variable_declaration.declared_type),
                                    /* antes: type_from_ast(...) */
                                    a->symbol_table.current_scope->level, ...);
    /* ... */
    //El simbolo ya es visible: int x = x; es valido.
    //El inicializador se verifica contra el tipo escrito en ESTA declaracion,
    //incluso si fue una redeclaracion.
    type_check_initialization(a, node);     /* antes: visit_expression(a,
    initializer) */
```

Código 17: `semantic.c`: cambios en `visit_variable_declaration`.

Por último, el caso por omisión de `visit_statement` (una expresión suelta) también pasa por `type_check_expression`. `visit_program`, `visit_block` y las funciones de `init`/`analyze`/`destroy` no cambiaron.

Funciones eliminadas

- `type_from_ast`: movida a `type_checker.c` con el nombre `type_from_declared`.
- `visit_identifier_use`: su trabajo (buscar el identificador y reportar si no existe) lo hace ahora `visit_identifier` en el verificador, que además devuelve el tipo.
- `visit_expression` (y su declaración adelantada): sustituida por `type_check_expression`. La anterior recorría la expresión solo para buscar identificadores, y no calculaba nada.

Propagación de `TYPE_ERROR` y prevención de cascadas

Esta es la parte donde el enunciado más insiste, así que dediquémosle espacio. El problema es el siguiente: si `x` no está declarada, `x + true` produce un primer error muy útil (“`x` no ha sido

declarada"). Si el verificador siguiera adelante y dijera además "el operador + requiere enteros", ese segundo mensaje sería solo consecuencia del primero y no aportaría nada.

Nuestra solución se apoya en tres piezas que ya vimos en el código:

1. **Un identificador no declarado devuelve `TYPE_ERROR`.** Es el origen del tipo "envenenado".
2. **Un operador con algún operando indeterminado devuelve `TYPE_ERROR` sin reportar nada.** Esto ocurre en `visit_unary_expression` y `visit_binary_expression`, antes de aplicar la regla. Así el error asciende por el árbol sin generar mensajes nuevos.
3. **Inicializaciones, asignaciones y condiciones no reportan si el tipo es indeterminado.** Es el `if (is_undetermined(actual)) return;` que se repite en las tres.

La propagación trata igual a `TYPE_ERROR` y a `TYPE_UNKNOWN`, un `TYPE_UNKNOWN` aparece, por ejemplo, cuando `type_check_expression` se interrumpe por un fallo interno; sin esta decisión, después de un error de memoria podrían aparecer incompatibilidades falsas.

La otra mitad del problema es no *ocultar* errores independientes, por eso:

- Los dos operandos de un binario se analizan siempre, aunque el primero ya sea `TYPE_ERROR`.
- El valor de una asignación y el inicializador de una declaración se analizan siempre, aunque el destino no exista o el nombre sea una redeclaración.
- Un error en la condición no impide analizar el cuerpo del `while` ni ninguna rama del `if`.

Veamos un caso real de nuestras pruebas, que reúne varias de estas situaciones, en cada línea hay exactamente un error *original*, y ninguna asignación produce un diagnóstico adicional por culpa de la suma incorrecta:

```
int n;
bool b;
n = b + 1;              // error en '+'; resultado TYPE_ERROR
b = n + true;           // error en '+'; no se compara la asignacion
n = (b && 1) + 2;       // error en '&&'; '+' y '=' callan
```

```
Error semantico [3:6]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [4:6]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.
Error semantico [5:7]:
el operador '&&' requiere operandos
de tipo 'bool', pero se encontraron
'bool' y 'int'.
```

Relación entre nodos y símbolos

El enunciado recomienda (aunque no obliga) que cada identificador del AST conserve una referencia al símbolo resuelto, y permite sustituirla por otra estrategia si queda claramente documentada. Decidimos no guardar apuntadores a símbolos en el AST, pues con la estrategia de liberación inmediata de la Práctica 5, `symbol_table_exit_scope` destruye los símbolos de un bloque al salir de él; cualquier `Symbol *` guardado en un nodo de ese bloque quedaría colgante en cuanto el bloque termina.

En su lugar, la relación entre un uso y su declaración se establece *durante* el recorrido, mediante la búsqueda visible, y lo que sobrevive en el árbol es el resultado que necesitan las etapas posteriores, el tipo (`inferred_type`), que es un valor. El símbolo conserva su nodo de declaración (`declaration`) y no es propietario de él, igual que antes.

Esto corrige una predicción de nuestro reporte anterior, donde anticipamos que en esta práctica tendríamos que conservar los ámbitos retirados, xomo no anotamos el árbol con apuntadores, no hace falta cambiar la estrategia de liberación. Si en una práctica posterior se necesitan esas referencias, el cambio sería conservar los ámbitos retirados y liberarlos en `symbol_table_destroy`; lo dejamos señalado como trabajo futuro.

Propiedad y liberación de memoria

En esta práctica no agregamos reservas de memoria nuevas, así que los propietarios siguen siendo los de la Práctica 5:

- **El símbolo** es propietario de la copia de su nombre no lo es de su nodo de declaración.
- **El ámbito** es propietario de sus símbolos y de su arreglo de buckets.
- **La tabla** es propietaria de los ámbitos de la pila vigente.
- **El analizador** es propietario de la tabla, pero no del AST, el árbol es de `main.c`, que lo libera con `ast_destroy`.
- **El campo** `inferred_type` es un valor dentro del nodo, no es propietario de nada y `ast_destroy` no cambia.
- **El verificador de tipos** no reserva memoria solo lee el AST y la tabla, y escribe un valor en cada nodo de expresión.

Como no hay apuntadores nuevos entre estructuras, el orden de destrucción sigue siendo el mismo (analizador, AST, parser y lexer) y no puede producir accesos inválidos. Verificamos la memoria con `valgrind` sobre los dos ejemplos del enunciado:

```
$ valgrind --leak-check=full --show-leak-kinds=all ./minic programa_valido.mc
Programa semanticamente correcto.
==17194== HEAP SUMMARY:
==17194==     in use at exit: 0 bytes in 0 blocks
==17194==   total heap usage: 122 allocs, 122 frees, 7,986 bytes allocated
```

```
==17194== All heap blocks were freed -- no leaks are possible
==17194== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ valgrind --leak-check=full --show-leak-kinds=all ./minic programa_invalido.mc
(seis diagnosticos en stderr)
==17228== HEAP SUMMARY:
==17228==     in use at exit: 0 bytes in 0 blocks
==17228==   total heap usage: 103 allocs, 103 frees, 6,626 bytes allocated
==17228== All heap blocks were freed -- no leaks are possible
==17228== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

En ambos casos todas las reservas se liberan y valgrind no reporta errores, incluida la ruta con errores semánticos.

Diagnósticos y recuperación

Formato de los errores

Los diagnósticos de tipos usan el mismo formato de la Práctica 5, la categoría, la posición entre corchetes (línea desde 1 y columna desde 0, igual que el lexer) y una descripción en varias líneas. Todos van a `stderr`, la posición que reportamos en cada caso es:

Construcción	**Posición del diagnóstico**
Operador binario o unario	El operador (`5 + true` se reporta en `[1:2]`).
Inicialización	El nodo del inicializador: para un literal o un identificador, su primer carácter (`int x = true;` en `[1:8]`).
Asignación	El identificador destino (`x = activo;` en la línea 4, `[4:0]`).
Condición de `if` / `while`	El nodo de la expresión condición.
Variable no declarada, redeclaración	Sin cambios respecto de la Práctica 5.

Table 2: Criterio de posición de los diagnósticos.

Para una inicialización con una expresión compuesta, como `int x = 1 < 2;`, el nodo del inicializador es el operador principal, así que la posición apunta al `<`. El enunciado permite reportar en el identificador, en el `=` o en el inicio de la expresión "siempre que el criterio sea consistente"; el nuestro es siempre la posición del nodo involucrado, y lo aplicamos igual en todas las construcciones.

Estos son los formatos de cada diagnóstico (el de variable no declarada y el de redeclaración son los de la Práctica 5):

```
Error semantico [1:15]:
no es posible inicializar una variable
de tipo 'int' con una expresion de tipo 'bool'.

Error semantico [4:0]:
no es posible asignar un valor de tipo 'bool'
```

```
a una variable de tipo 'int'.

Error semantico [1:2]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.

Error semantico [1:13]:
el operador '==' requiere operandos
del mismo tipo, pero se encontraron
'bool' y 'int'.

Error semantico [1:0]:
el operador unario '-' requiere
un operando de tipo 'int',
pero se encontro 'bool'.

Error semantico [2:4]:
la condicion de una sentencia 'if'
debe ser de tipo 'bool',
pero se encontro 'int'.
```

Acumulación de errores

Igual que en la práctica anterior, un error semántico no detiene el análisis, el verificador reporta, devuelve `TYPE_ERROR` y el recorrido continúa, así que se detectan varios errores a la vez, siempre en el orden del programa. El ejemplo del enunciado lo muestra bien, con `x = activo; activo = 10; print(x + true);` se reportan las tres incompatibilidades, y ninguna modifica el tipo de `x` ni el de `activo`.

También conservamos la regla de que el análisis de tipos *solo se ejecuta sobre una estructura sintáctica válida* que, ante un error léxico o sintáctico, `main.c` no llama al analizador semántico y, por lo tanto, tampoco al verificador.

Errores internos

El mecanismo no cambió, pero conviene recorrerlo con las preguntas que nos plantea el enunciado para revisar el diseño:

- **¿Quién comunica el fallo interno?** `semantic_report_internal_error`. Escribe el mensaje por `stderr` una única vez y activa `had_internal_error`. Después, `semantic_analyze` devuelve `SEM_INTERNAL_ERROR` y `main.c` lo traduce al código de salida `2`. Nunca se presenta como incompatibilidad de tipos: `TYPE_ERROR` solo significa "una expresión afectada por un error semántico", no "algo falló".
- **¿Quién posee cada estructura?** Lo detallamos en la sección de memoria, el símbolo, su nombre, el ámbito, sus símbolos, la tabla, los ámbitos, el analizador, la tabla; y `main.c`, el AST.

- **¿Dónde se evita duplicar diagnósticos?** El recorrido único (cada uso se visita una vez), la propagación de `TYPE_ERROR` (los padres callan) y el hecho de que el destino de una asignación lo resuelve `semantic.c` y lo reporta una sola vez (el verificador solo recibe el resultado).
- **¿Cómo se respeta el *shadowing*?** El tipo de un identificador sale de `symbol_table_lookup`, que va del ámbito actual al global y devuelve la declaración más cercana. Como la búsqueda ocurre en el momento de la visita, siempre refleja el ámbito vigente.
- **¿En qué momento se escribe el mensaje de éxito?** Únicamente en `main.c`, dentro de `analyze`, en el caso `SEM_OK` y *después* de destruir el analizador. Nada se imprime en `stdout` mientras el análisis está en marcha, por lo que un programa con errores deja `stdout` vacío.

Resultados y pruebas

Cambios en el `Makefile`

- Agregamos `src/semantic/type.c` y `src/semantic/type_checker.c` a `SOURCES`. Los encabezados nuevos ya quedan cubiertos por el `wildcard` de `HEADERS`.

```
SOURCES = src/main.c \
src/lexer/token.c src/lexer/keywords.c \
src/lexer/lexer.c src/lexer/buffer_manage.c \
src/parser/parser.c src/parser/grammar.c \
src/ast/ast.c \
src/semantic/symbol.c src/semantic/scope.c \
src/semantic/symbol_table.c src/semantic/type.c \
src/semantic/type_checker.c src/semantic/semantic.c
```

- Retiramos los objetivos y variables de las pruebas públicas de las Prácticas 1 a 5 (`test-p1` a `test-p5`), para no extender innecesariamente los archivos del entregable. Además agregamos `test-p6`, que ejecuta el script oficial de la práctica, `test-cli` ahora solo depende de `test-p6`.

```
PUBLIC_P6_DIR = tests/public/p6
PUBLIC_P6_SCRIPT = $(PUBLIC_P6_DIR)/ejecutar_pruebas.sh

test-p6: $(TARGET)
@echo "==== public-p6 ===="
@sh $(PUBLIC_P6_SCRIPT) ./$(TARGET)
```

- Conservamos las pruebas propias (`test-unit-lexer`, `test-unit-parser`, `test-unit-ast`, `test-unit-semantic`) y `test-args`, sin cambios.

Pruebas públicas de la Práctica 6

Para ejecutar estas pruebas, después de compilar, usamos:

```
$ make test-p6
```

Esta es la salida completa:

```
==== public-p6 ====
[CONTRATO OK] p01_inicializacion_valida

Diagnosticos de p02_inicializacion_invalida (revisar CASOS.md):
Error semantico [1:8]:
no es posible inicializar una variable
de tipo 'int' con una expresion de tipo 'bool'.
Error semantico [2:9]:
no es posible inicializar una variable
de tipo 'bool' con una expresion de tipo 'int'.
[CONTRATO OK] p02_inicializacion_invalida
[CONTRATO OK] p03_asignacion_valida

Diagnosticos de p04_asignacion_invalida (revisar CASOS.md):
Error semantico [3:0]:
no es posible asignar un valor de tipo 'bool'
a una variable de tipo 'int'.
Error semantico [4:0]:
no es posible asignar un valor de tipo 'int'
a una variable de tipo 'bool'.
[CONTRATO OK] p04_asignacion_invalida
[CONTRATO OK] p05_aritmetica_valida

Diagnosticos de p06_aritmetica_invalida (revisar CASOS.md):
Error semantico [1:11]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [2:12]:
el operador '-' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [3:8]:
el operador '*' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.
Error semantico [4:12]:
el operador '/' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
```

```
Error semantico [5:6]:
el operador unario '-' requiere
un operando de tipo 'int',
pero se encontro 'bool'.
[CONTRATO OK] p06_aritmetica_invalida
[CONTRATO OK] p07_relacionales_validos

Diagnosticos de p08_relacionales_invalidos (revisar CASOS.md):
Error semantico [1:11]:
el operador '<' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [2:8]:
el operador '<=' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.
Error semantico [3:11]:
el operador '>' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'bool'.
Error semantico [4:12]:
el operador '>=' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
[CONTRATO OK] p08_relacionales_invalidos
[CONTRATO OK] p09_igualdad_valida

Diagnosticos de p10_igualdad_invalida (revisar CASOS.md):
Error semantico [1:8]:
el operador '==' requiere operandos
del mismo tipo, pero se encontraron
'int' y 'bool'.
Error semantico [2:12]:
el operador '!=' requiere operandos
del mismo tipo, pero se encontraron
'bool' y 'int'.
[CONTRATO OK] p10_igualdad_invalida
[CONTRATO OK] p11_logicos_validos

Diagnosticos de p12_logicos_invalidos (revisar CASOS.md):
Error semantico [1:8]:
el operador '&&' requiere operandos
de tipo 'bool', pero se encontraron
'int' y 'bool'.
Error semantico [2:12]:
el operador '||' requiere operandos
de tipo 'bool', pero se encontraron
```

```
'bool' y 'int'.
[CONTRATO OK] p12_logicos_invalidos
[CONTRATO OK] p13_control_valido

Diagnosticos de p14_control_invalido (revisar CASOS.md):
Error semantico [1:4]:
la condicion de una sentencia 'if'
debe ser de tipo 'bool',
pero se encontro 'int'.
Error semantico [1:20]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [2:7]:
la condicion de una sentencia 'while'
debe ser de tipo 'bool',
pero se encontro 'int'.
Error semantico [2:18]:
el operador unario '-' requiere
un operando de tipo 'int',
pero se encontro 'bool'.
[CONTRATO OK] p14_control_invalido

Diagnosticos de p15_ramas_no_ejecutadas (revisar CASOS.md):
Error semantico [1:19]:
la variable 'ausente' no ha sido declarada.
Error semantico [1:44]:
el operador unario '-' requiere
un operando de tipo 'int',
pero se encontro 'bool'.
Error semantico [2:24]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.
[CONTRATO OK] p15_ramas_no_ejecutadas
[CONTRATO OK] p16_shadowing

Diagnosticos de p17_orden_visibilidad (revisar CASOS.md):
Error semantico [1:6]:
la variable 'x' no ha sido declarada.
[CONTRATO OK] p17_orden_visibilidad

Diagnosticos de p18_redeclaracion (revisar CASOS.md):
Error semantico [2:5]:
la variable 'x' ya fue declarada
en este ambito.
Declaracion anterior [1:4].
```

```
[CONTRATO OK] p18_redeclaracion

Diagnosticos de p19_destino_ausente (revisar CASOS.md):
Error semantico [1:0]:
la variable 'ausente' no ha sido declarada.
Error semantico [1:15]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
[CONTRATO OK] p19_destino_ausente

Diagnosticos de p20_propagacion (revisar CASOS.md):
Error semantico [1:13]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [2:10]:
el operador '*' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [3:6]:
la variable 'no_declarada' no ha sido declarada.
[CONTRATO OK] p20_propagacion

Diagnosticos de p21_hermanos_independientes (revisar CASOS.md):
Error semantico [1:12]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
Error semantico [1:26]:
el operador '*' requiere operandos
de tipo 'int', pero se encontraron
'bool' y 'int'.
[CONTRATO OK] p21_hermanos_independientes
[CONTRATO OK] p22_completo_valido

Diagnosticos de p23_error_lexico (revisar CASOS.md):
Error lexico [1:8]: caracter invalido '@'.
Error sintactico [1:9]: se esperaba una expresion, pero se encontro ';'.
[CONTRATO OK] p23_error_lexico

Diagnosticos de p24_error_sintactico (revisar CASOS.md):
Error sintactico [1:8]: se esperaba una expresion, pero se encontro ';'.
[CONTRATO OK] p24_error_sintactico

Contratos: 24 correctos, 0 fallidos. Diagnosticos pendientes de revision humana.
```

Revisemos algunas salidas porque muestran las decisiones de diseño en acción:

- **p14.** Se reportan el error de la condición del `if` (`[1:4]`) *y* el del `+` del cuerpo (`[1:20]`); lo mismo con el `while` (`[2:7]` y `[2:18]`). Un error en la condición no impide analizar el interior.
- **p15.** En las ramas aparecen una variable no declarada, una negación y una suma incorrectas, los tres independientes.
- **p19.** Con `ausente` sin declarar, el programa produce la variable no declarada (`[1:0]`) y el error del `+` del valor (`[1:15]`), pero ninguna incompatibilidad de asignación.
- **p20.** Después de cada error de operador, las asignaciones y expresiones que lo contienen callan; solo aparecen los errores de origen (dos operadores y una variable no declarada).
- **p23 y p24.** Solo hay errores léxicos y sintácticos: `main.c` no llama al analizador semántico, y por lo tanto tampoco al verificador de tipos.

Además, ejecutamos los dos programas de ejemplo que acompañan al enunciado. El programa válido imprime el mensaje de éxito con código de salida `0` y `stderr` vacío. El inválido produce seis diagnósticos, que coinciden con la salida de ejemplo del enunciado (sección 34), con `stdout` vacío y código de salida `1`:

```
Error semantico [1:15]:
no es posible inicializar una variable
de tipo 'int' con una expresion de tipo 'bool'.
Error semantico [2:14]:
no es posible inicializar una variable
de tipo 'bool' con una expresion de tipo 'int'.
Error semantico [4:0]:
no es posible asignar un valor de tipo 'bool'
a una variable de tipo 'int'.
Error semantico [6:4]:
la condicion de una sentencia 'if'
debe ser de tipo 'bool',
pero se encontro 'int'.
Error semantico [10:7]:
la condicion de una sentencia 'while'
debe ser de tipo 'bool',
pero se encontro 'int'.
Error semantico [11:22]:
el operador '+' requiere operandos
de tipo 'int', pero se encontraron
'int' y 'bool'.
```

Es una buena comprobación porque `[11:22]` corresponde al operador `+` de `contador + false`: confirma que el parser guarda en los nodos binarios la columna del operador, que es lo que suponen nuestros diagnósticos. Además, `activo = contador + false;` produce *solo* el error del `+`, y no una incompatibilidad de asignación adicional: es la propagación de `TYPE_ERROR` en acción.

Pruebas propias del analizador semántico

¿Qué pasó con las pruebas de la Práctica 5?

Le pasamos a Claude las pruebas de la Práctica 5 para que corroborara que funcionan según los nuevos objetivos de la Práctica 6, nos confirmó que siguen siendo válidas, nos dio casos extra, y todos funcionan. Temíamos que, al validar tipos, las 40 pruebas anteriores dejaran de pasar; Claudio nos explicó que la razón por la que no es así es que en todas las variables que reciben valores son `int` y se les asignan expresiones enteras, y los `bool` aparecen solo con `true`, `false` o como condición. Son programas correctos también para el sistema de tipos, y prueban cosas que la Práctica 6 exige conservar (ámbitos, *shadowing*, redeclaración, uso antes de declarar y recuperación ante varios errores).

Ejecución de las pruebas

Para ejecutar nuestras pruebas de esta práctica, es necesario utilizar el siguiente comando después de compilar:

```
$ make test-unit-semantic
```

El sistema ejecuta cada archivo `.mc` y compara la salida estándar, los diagnósticos y el código de salida contra los archivos esperados; una prueba es correcta solo si los tres coinciden. Obtuvimos:

```
==== unit-semantic ====
PASS: ut_sem_assignment_target
PASS: ut_sem_block_redeclaration
PASS: ut_sem_condition_use
PASS: ut_sem_deep_nesting
PASS: ut_sem_empty_blocks
PASS: ut_sem_empty
PASS: ut_sem_errors_inside_blocks
PASS: ut_sem_expression_use
PASS: ut_sem_global_from_nested
PASS: ut_sem_global_redeclaration
PASS: ut_sem_if_else_valid
PASS: ut_sem_if_scopes
PASS: ut_sem_if_without_blocks_ok
PASS: ut_sem_initializer_uses
PASS: ut_sem_long_identifier_redecl
PASS: ut_sem_long_identifiers
...
PASS: ut_sem_type_while_invalid
PASS: ut_sem_type_while_valid
PASS: ut_sem_undeclared_completely
PASS: ut_sem_use_after_declaration
PASS: ut_sem_use_before_declaration
PASS: ut_sem_while_local_escape
PASS: ut_sem_while_locals
```

```
----
Total: 81  Pass: 81  Fail: 0
```

Los 81 casos pasan, los 40 de la Práctica 5, sin modificarlos, y los 41 nuevos.

Los 41 casos nuevos de la Práctica 6

Los casos que agregamos (por parte de Claude) son propios de la verificación de tipos, llevan el prefijo `ut_sem_type_` (que omitimos en las tablas) y siguen la lista de temas mínimos del enunciado. Cada programa `.mc` tiene su salida estándar esperada (`.out`), sus diagnósticos esperados (`.err`) y su código de salida (`.code`), igual que los anteriores.

Literales, identificadores e inicialización.

Caso	Qué verifica	Resultado
`int_literal`	`print` de enteros, incluido uno negativo.	Correcto
`bool_literal`	`print(true)` y `print(false)`.	Correcto
`identifiers`	Un identificador toma el tipo de su declaración.	Correcto
`init_valid`	Inicializadores `int` y `bool`, también con expresiones y variables.	Correcto
`init_invalid`	`int` inicializado con `false` y `bool` con `1`.	2 errores
`redeclaration_init`	`int x; int x = true; bool x = 1;`: cada redeclaración da el error de redeclaración y el de inicialización, medido contra el tipo escrito en esa declaración.	4 errores

Table 3: Literales, identificadores e inicialización (prefijo `ut_sem_type_`).

Asignación.

Caso	Qué verifica	Resultado
`assign_valid`	Asignaciones `int` y `bool` con expresiones del mismo tipo.	Correcto
`assign_invalid`	`x = activo;` y `activo = 10;`	2 errores
`assignment_missing_target`	`y = true + 1;` y `z = w;` con destinos no declarados: se reportan la variable y el operador, pero ninguna incompatibilidad de asignación.	4 errores

Table 4: Asignación (prefijo `ut_sem_type_`).

Operadores.

Caso	Qué verifica	Resultado
`arith_valid`	Las cuatro operaciones con enteros, mezcladas.	Correcto
`arith_invalid`	`+ * - /` con algún operando `bool`.	4 errores
`unary_valid`	Negación de literales y variables, incluida `-contador`.	Correcto
`unary_invalid`	`-true` y `-b` con `b` booleana.	2 errores
`relational_valid`	`< <= > >=` entre enteros y su asignación a `bool`.	Correcto
`relational_invalid`	Relacionales con operandos `bool`.	3 errores
`equality_valid`	`== !=` entre dos enteros y entre dos booleanos.	Correcto
`equality_invalid`	`5 == true` y `activo != 10`: tipos distintos.	2 errores
`logical_valid`	`&&` y `||` con booleanos, incluida una mezcla con relacionales.	Correcto
`logical_invalid`	`3 && 5` y `activo || 10`.	2 errores

Table 5: Operadores (prefijo `ut_sem_type_`).

Condiciones y sentencias.

Caso	Qué verifica	Resultado
`if_valid`	Condiciones booleanas, también comparaciones.	Correcto
`if_invalid`	`if (10)` y `if (x)` con `x` entera: un entero no es verdadero.	2 errores
`while_valid`	Ciclo con condición relacional.	Correcto
`while_invalid`	`while (5)` y `while (contador)`.	2 errores
`branches_still_analyzed`	`if (5)` y `while (7)` con cuerpos que tienen una variable inexistente y asignaciones incompatibles: el error de la condición no oculta los del interior.	6 errores
`print_any_type`	`print` acepta enteros, booleanos y expresiones de ambos tipos.	Correcto

Table 6: Condiciones y sentencias (prefijo `ut_sem_type_`).

Expresiones compuestas.

Caso	Qué verifica	Resultado
`nested_expressions`	Paréntesis, negación y combinaciones aritméticas, relacionales y lógicas bien tipadas.	Correcto
`precedence_valid`	Precedencia entre aritméticos, relacionales, igualdad y lógicos.	Correcto
`precedence_invalid`	`1 + 2 == 3 < 4` se agrupa como `(1 + 2) == (3 < 4)`: compara `int` con `bool`.	1 error [1:12]
`associativity_invalid`	`1 == 1 == 1` se agrupa como `(1 == 1) == 1`.	1 error [1:13]

Table 7: Expresiones compuestas (prefijo `ut_sem_type_`).

Propagación y cascadas.

Caso	Qué verifica	Resultado
`error_propagation`	Cinco expresiones con una variable inexistente dentro de operadores y condiciones: solo deben aparecer errores de variable no declarada.	5 errores
`cascade_assignment`	Un operador incorrecto dentro de una asignación: no debe aparecer una incompatibilidad de asignación derivada.	3 errores
`multiple_errors`	Dos asignaciones incompatibles y un operador incorrecto: se acumulan sin alterar los tipos de las variables.	3 errores
`independent_siblings`	`-true + (false < true)`: los errores de los dos subárboles son independientes y se reportan ambos, sin uno extra del `+`.	2 errores

Table 8: Propagación y cascadas (prefijo `ut_sem_type_`).

Ámbitos con tipos.

Caso	Qué verifica	Resultado
`shadowing_valid`	Una `bool x` interna oculta a una `int x`; `if (x)` dentro es válido y `x = 10` fuera también.	Correcto
`shadowing_inner_int`	Una `int x` interna oculta a una `bool x`; fuera del bloque `if (x)` vuelve a ser válido.	Correcto
`shadowing_invalid`	Las mismas variables pero con asignaciones y condición del tipo del ámbito equivocado.	3 errores
`scope_and_type_errors`	Mezcla de una incompatibilidad, una variable fuera de su bloque y variables inexistentes.	4 errores

Table 9: Ámbitos con tipos (prefijo `ut_sem_type_`).

Programas completos.

Caso	Qué verifica	Resultado
`spec_example_valid`	Ejemplo válido del enunciado.	Correcto
`spec_example_invalid`	Ejemplo inválido del enunciado: dos inicializaciones, una condición y un operador.	4 errores
`complete_valid`	Programa con `while`, `if` y `print` bien tipados.	Correcto
`complete_invalid`	Programa de la sección 34 del enunciado, con sus seis diagnósticos.	6 errores

Table 10: Programas completos (prefijo `ut_sem_type_`).

Problemas encontrados

Ciclo de inclusión entre `ast.h` y `symbol.h`

Para guardar `Type` dentro del nodo, `ast.h` necesita conocerlo; pero `Type` estaba en `symbol.h`, que ya incluye a `ast.h`. Era un ciclo que resolvimos moviendo `Type` a `type.h`, un encabezado sin dependencias del proyecto, que ahora incluyen los dos. De ahí viene también que la conversión `ASTDeclaredType` → `Type` viva en `type_checker.h` y no en `type.h`.

La declaración y la definición de `semantic_report_error` no coincidían

Al compilar por primera vez obtuvimos un error de "too many arguments" en todas las llamadas desde `type_checker.c` y, en `semantic.c`, "conflicting types". La causa era que en `semantic.h` habíamos escrito la declaración sin los `...`, y en C una función con número variable de argumentos y otra sin él son tipos distintos. Corregimos la declaración y el error desapareció.

¿Guardar el símbolo en el AST?

La recomendación del enunciado era el objetivo inicial, pero con la liberación inmediata de ámbitos esos apuntadores quedarían colgantes. Preferimos no guardarlos y documentar la decisión (sección *Relación entre nodos y símbolos*). Es una decisión de tiempos de vida, y la tomamos conscientes de que limita lo que podrán hacer las etapas posteriores sin volver a buscar el símbolo.

Dónde medir la posición de cada error

Los ejemplos del enunciado suponían que un error de operador se reporta en la columna del operador, y no en la del primer operando. Como no modificamos el parser en esta práctica, necesitábamos confirmar que los nodos binarios y unarios guardaban esa posición. La salida de `programa_invalido.mc` (`[11:22]`) lo confirmó para los binarios.

Límite por anidamiento profundo

Como en las prácticas anteriores, el recorrido es recursivo; ahora también el verificador de tipos. Una entrada con una cantidad extrema de paréntesis o de bloques anidados puede agotar la pila.

No agregamos un límite de profundidad y lo dejamos registrado como limitación conocida.

Conclusiones

En esta práctica vimos que verificar tipos es, en buena medida, repetir con otra información el mismo recorrido que ya hacíamos. El AST y la tabla de símbolos de las prácticas anteriores tenían casi todo lo necesario, así que no tocamos el lexer ni el parser, y en el árbol solo agregamos un campo. Seguimos confirmando que separar las etapas nos permite agregar una nueva sin reescribir las anteriores.

La idea que más pesó en el diseño fue `TYPE_ERROR`. Con un solo valor especial, que se genera en el origen del error y se propaga hacia arriba sin decir nada, obtuvimos dos propiedades que parecen opuestas: no repetir diagnósticos derivados y, al mismo tiempo, no ocultar errores independientes. Para lograrlo hubo que ser disciplinados en un punto concreto: analizar *siempre* los dos operandos, el valor de una asignación y las ramas de un condicional, y decidir si reportar o no solo después.

También aprendimos a tomar decisiones por tiempos de vida y no por comodidad. Guardar el símbolo resuelto en cada identificador era lo que el enunciado recomendaba, pero choca con liberar los símbolos al salir de un bloque. Preferimos conservar solo el tipo, que es un valor, y dejar señalado qué habría que cambiar si una etapa posterior necesitara los símbolos.

Por último, las pruebas nos dieron dos cosas. Primero, la tranquilidad de comprobar, en lugar de suponer, que las 40 pruebas anteriores seguían valiendo. Segundo, casos nuevos que verifican exactamente los puntos delicados de la práctica (cascada, errores independientes, *shadowing* con tipos, redeclaración con inicializador), con esperados contrastados con los ejemplos del enunciado. Como limitaciones, el diseño sigue siendo recursivo, y las posiciones de los errores de inicialización con expresiones compuestas apuntan al operador principal, criterio que mantuvimos consistente y documentado.

División del trabajo

Para esta práctica, el trabajo tanto del código como del reporte fue realizado en conjunto por ambos miembros del equipo durante tiempos libres dentro de la facultad. Discutiendo y analizando lo necesario a implementar, diseñar y cómo realizarlo. Nos reunimos en el laboratorio del Tlahuizcalpan *Taller de Lenguajes Simbólicos*.

Uso de herramientas de inteligencia artificial

Como mencionamos anteriormente, los casos nuevos propios de la verificación de tipos (`ut_sem_type_*`) fueron proporcionados por Claude. Le pasamos el contexto de nuestro reporte y un zip con las pruebas anteriores para la Práctica 5. Además del resumen del reporte mismo para el CHANGELOG.md

References

[1] cppreference.com. (n.d.). C language. Disponible en https://en.cppreference.com/c/language

[2] cppreference.com. (n.d.). C11. Disponible en https://en.cppreference.com/c/11

[3] Free Software Foundation. (n.d.). GNU make. Disponible en https://www.gnu.org/software/make/manual/make.html